



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 11.a: Full-Stack con Spring Boot (H2 Database) y Angular (Reactive Forms)

Resumen

Esta práctica guiada tiene como objetivo instruir paso a paso en la creación de un ecosistema Full-Stack completo. Se desarrollará un sistema de **Registro de Charlas para una Conferencia Tecnológica (TechConf)**.

En el **Back-End (Spring Boot)**, aprenderá a configurar y habilitar una base de datos en memoria **H2**, a trabajar con entidades JPA y a inicializar la base de datos con datos semilla utilizando el archivo `data.sql`. En el **Front-End (Angular Standalone)**, expandiremos el conocimiento sobre formularios introduciendo **Reactive Forms**, el enfoque recomendado por Angular para manejar formularios complejos y escalables mediante validaciones programáticas directas en el controlador TypeScript.

Metadatos del Laboratorio

- **Tiempo estimado:** 3.5 horas.
- **Herramientas requeridas:** Java 17+, Maven, Node.js 18+, Angular CLI (@angular/cli), IDE (VS Code o IntelliJ), Navegador Web.

Metas de Aprendizaje:

1. Configurar H2 Database en Spring Boot y acceder a su consola web.
2. Poblar la base de datos automáticamente en tiempo de arranque con un script `data.sql`.
3. Construir un API REST funcional para listar y guardar entidades.
4. Implementar un formulario en Angular utilizando `ReactiveFormsModule` (`FormGroup`, `FormControl`).
5. Aplicar y mostrar validaciones síncronas de Angular en tiempo real (campos requeridos, formato de correo, longitud mínima).

Parte 1: Construcción del Back-End (Spring Boot y H2)

Paso 1.1: Generación del Proyecto y Configuración de H2

1. Diríjase a [Spring Initializr](#).
2. Configure el proyecto: Maven, Java 17+, Packaging Jar.
3. Agregue las dependencias: **Spring Web**, **Spring Data JPA**, y **H2 Database**.
4. Descargue, descomprima y abra el proyecto en su IDE.

Abra el archivo `src/main/resources/application.properties` y agregue la siguiente configuración para habilitar H2, establecer sus credenciales y permitir la ejecución del script SQL inicial:

```
# Configuración del puerto del servidor
server.port=8080
```

```
# Configuración de H2 Database (En memoria)
spring.datasource.url=jdbc:h2:mem:techconfdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Habilitar la consola web de H2
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Evitar que Hibernate sobrescriba nuestros inserts manuales,
# y forzar que se ejecute el data.sql siempre
spring.jpa.hibernate.ddl-auto=update
spring.sql.init.mode=always
```

Paso 1.2: Creación de la Semilla de Datos (data.sql)

Spring Boot detecta automáticamente un archivo llamado `data.sql` en la carpeta `resources` y lo ejecuta al levantar la aplicación. Esto es extremadamente útil para tener datos de prueba (seeds) en una base de datos en memoria.

1. Cree un archivo llamado `data.sql` en la ruta `src/main/resources/data.sql`.
2. Agregue el siguiente código SQL:

```
INSERT INTO charla (titulo, expositor, nivel, email_contacto) VALUES ('Introduccion a la Inteligencia Artificial')
INSERT INTO charla (titulo, expositor, nivel, email_contacto) VALUES ('Microservicios con Spring Cloud')
INSERT INTO charla (titulo, expositor, nivel, email_contacto) VALUES ('Angular 18: Señales y Standalone Components')
```

Paso 1.3: Modelo de Dominio y Repositorio

Cree la entidad `Charla`:

```
package com.techconf.models;

import jakarta.persistence.*;

@Entity
public class Charla {
```

```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String titulo;

    @Column(nullable = false)
    private String expositor;

    private String nivel; // Principiante, Intermedio, Avanzado

    @Column(name = "email_contacto")
    private String emailContacto;

    // Constructores, Getters y Setters
    public Charla() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public String getExpositor() { return expositor; }
    public void setExpositor(String expositor) { this.expositor = expositor; }
    public String getNivel() { return nivel; }
    public void setNivel(String nivel) { this.nivel = nivel; }
    public String getEmailContacto() { return emailContacto; }
    public void setEmailContacto(String emailContacto) { this.emailContacto = emailContacto; }
}

```

Cree el repositorio CharlaRepository:

```

package com.techconf.repositories;

import com.techconf.models.Charla;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface CharlaRepository extends JpaRepository<Charla, Long> {
}

```

Paso 1.4: Controlador REST

Cree el controlador para exponer los datos y guardar nuevos registros:

```
package com.techconf.controllers;

import com.techconf.models.Charla;
import com.techconf.repositories.CharlaRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/charlas")
@CrossOrigin(origins = "http://localhost:4200")
public class CharlaController {

    @Autowired
    private CharlRepository repository;

    @GetMapping
    public List<Charla> obtenerTodas() {
        return repository.findAll();
    }

    @PostMapping
    public Charla registrarCharla(@RequestBody Charla nuevaCharla) {
        return repository.save(nuevaCharla);
    }
}
```

Verificación del Backend:

Ejecute el proyecto Spring Boot. Ingrese a <http://localhost:8080/h2-console> en su navegador. Use la URL JDBC `jdbc:h2:mem:techconfdb`, usuario `sa` y contraseña `password`. Verá la tabla CHARLA con los 3 registros poblados por su `data.sql`.

Parte 2: Desarrollo del Front-End (Angular y Reactive Forms)

A diferencia de *Template-Driven Forms* (donde la lógica reside en el HTML vía `[(ngModel)]`), **Reactive Forms** define la estructura y validaciones del formulario directamente en la clase TypeScript, otorgando un control absoluto y predecible del estado del formulario, ideal para

aplicaciones complejas.

Paso 2.1: Inicialización del Proyecto

En su consola de comandos, ejecute:

```
ng new techconf-frontend --standalone
```

(Seleccione CSS y presione No a SSR).

En `src/app/app.config.ts`, habilite el cliente HTTP para conectarnos al backend:

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withFetch())
  ]
};
```

Paso 2.2: Servicio de Comunicación HTTP

Cree el modelo y el servicio para la comunicación con Spring Boot:

```
ng generate service services/charla
```

src/app/services/charla.service.ts:

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';

export interface Charla {
  id?: number;
  titulo: string;
}
```

```

    expositor: string;
    nivel: string;
    emailContacto: string;
  }

  @Injectable({
    providedIn: 'root'
  })
  export class CharlaService {
    private http = inject(HttpClient);
    private apiUrl = 'http://localhost:8080/api/charlas';

    getCharlas(): Observable<Charla[]> {
      return this.http.get<Charla[]>(this.apiUrl);
    }

    registrarCharla(charla: Charla): Observable<Charla> {
      return this.http.post<Charla>(this.apiUrl, charla);
    }
  }

```

Paso 2.3: Componente con Reactive Forms (TypeScript)

Genere el componente principal donde vivirá el formulario:

```
ng generate component components/charla-registro --standalone
```

charla-registro.component.ts: Aquí reside el núcleo de Reactive Forms. Importamos `ReactiveFormsModule` y utilizamos `FormGroup`, `FormControl` y `Validators`.

```

import { Component, inject, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ReactiveFormsModule, FormGroup, FormControl, Validators } from '@angular/forms';
import { CharlaService, Charla } from '../../services/charla.service';

@Component({
  selector: 'app-charla-registro',
  standalone: true,
  // ¡IMPORTANTE! Se debe importar ReactiveFormsModule para activar Reactive Forms
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './charla-registro.component.html',
  styleUrls: ['./charla-registro.component.css']
})

```

```

})
export class CharlaRegistroComponent implements OnInit {
  private charlaService = inject(CharlaService);

  charlas: Charla[] = [];
  mensajeExito: string = '';

  // 1. Definición de la estructura y validadores del formulario reactivo
  registroForm = new FormGroup({
    titulo: new FormControl('', [Validators.required, Validators.minLength(5)]),
    expositor: new FormControl('', [Validators.required]),
    nivel: new FormControl('Principiante', [Validators.required]),
    emailContacto: new FormControl('', [Validators.required, Validators.email])
  });

  ngOnInit(): void {
    this.cargarCharlas();
  }

  cargarCharlas() {
    this.charlaService.getCharlas().subscribe({
      next: (data) => this.charlas = data,
      error: (err) => console.error("Error al cargar las charlas", err)
    });
  }

  // 2. Método de envío
  onSubmit(): void {
    // Si el formulario es inválido, forzamos a mostrar los errores visualmente
    if (this.registroForm.invalid) {
      this.registroForm.markAllAsTouched();
      return;
    }

    // Extraemos los valores ya tipados y validados del formulario
    const nuevaCharla: Charla = this.registroForm.value as Charla;

    this.charlaService.registrarCharla(nuevaCharla).subscribe({
      next: (res) => {
        this.mensajeExito = '¡Charla registrada exitosamente!';
        this.charlas.push(res);
        // Resetea el formulario dejándolo en estado prístino
        this.registroForm.reset({ nivel: 'Principiante' });
      },
      error: (err) => console.error(err)
    });
  }

```

```

}

// Getters auxiliares para facilitar el acceso en la vista HTML
get tituloCtrl() { return this.registroForm.get('titulo'); }
get expositorCtrl() { return this.registroForm.get('expositor'); }
get emailCtrl() { return this.registroForm.get('emailContacto'); }
}

```

Paso 2.4: Vista del Componente (HTML)

charla-registro.component.html: La directiva `[formGroup]="registroForm"` enlaza el elemento `<form>` del HTML con nuestro objeto TS. A su vez, `formControlName="..."` enlaza cada input individual con su respectivo `FormControl`.

```

<div class="container">
  <div class="form-section">
    <h2>Registrar Nueva Charla</h2>

    @if(mensajeExito) {
      <div class="alert success">{{ mensajeExito }}</div>
    }

    <!-- Vinculación del FormGroup -->
    <form [formGroup]="registroForm" (ngSubmit)="onSubmit()">

      <div class="form-group">
        <label>Título de la Charla:</label>
        <input type="text" formControlName="titulo" placeholder="Ej. Arquitectura Hexagonal">
        <!-- Mostrar errores si el campo es inválido y fue tocado por el usuario -->
        @if(tituloCtrl?.invalid && (tituloCtrl?.dirty || tituloCtrl?.touched)) {
          <div class="error">
            @if(tituloCtrl?.hasError('required')) { <span>El título es requerido.</span> }
            @if(tituloCtrl?.hasError('minlength')) { <span>El título debe tener al menos 5 caracteres.</span> }
          </div>
        }
      </div>

      <div class="form-group">
        <label>Expositor:</label>
        <input type="text" formControlName="expositor" placeholder="Nombre completo">
        @if(expositorCtrl?.invalid && expositorCtrl?.touched) {
          <div class="error">El expositor es requerido.</div>
        }
      </div>
    </form>
  </div>
</div>

```

```

    <div class="form-group">
      <label>Nivel de la Charla:</label>
      <select formControlName="nivel">
        <option value="Principiante">Principiante</option>
        <option value="Intermedio">Intermedio</option>
        <option value="Avanzado">Avanzado</option>
      </select>
    </div>

    <div class="form-group">
      <label>Email de Contacto:</label>
      <input type="email" formControlName="emailContacto" placeholder="correo@ejemplo.com">
      @if(emailCtrl?.invalid && emailCtrl?.touched) {
        <div class="error">
          @if(emailCtrl?.hasError('required')) { <span>El email es requerido.</span> }
          @if(emailCtrl?.hasError('email')) { <span>Formato de email inválido.</span> }
        </div>
      }
    </div>

    <!-- El botón se desactiva automáticamente si el formulario es inválido (opcional) -->
    <button type="submit" [disabled]="registroForm.invalid" class="submit-btn">Guardar Charla</button>
  </form>
</div>

<div class="list-section">
  <h2>Agenda de TechConf</h2>
  <div class="card-grid">
    @for(charla of charlas; track charla.id) {
      <div class="card">
        <h3>{{ charla.titulo }}</h3>
        <p><strong>Expositor:</strong> {{ charla.expositor }}</p>
        <p><strong>Nivel:</strong> <span class="badge">{{ charla.nivel }}</span></p>
        <p><strong>Contacto:</strong> {{ charla.emailContacto }}</p>
      </div>
    } @empty {
      <p>No hay charlas registradas aún en el sistema.</p>
    }
  </div>
</div>
</div>

```

(Recuerde declarar el componente `<app-charla-registro></app-charla-registro>` en el `app.component.html` o enrutarlo vía `app.routes.ts` para poder visualizarlo en el navegador).

¡Felicidades! Ha finalizado la Práctica Guiada 11.a. En ella, ha logrado construir un ecosistema completo que no solo expone y guarda datos desde una base de datos embebida (H2) poblada automáticamente con semillas (`data.sql`), sino que además recolecta datos del usuario de forma sumamente controlada mediante el modelo avanzado de **Reactive Forms** en Angular.